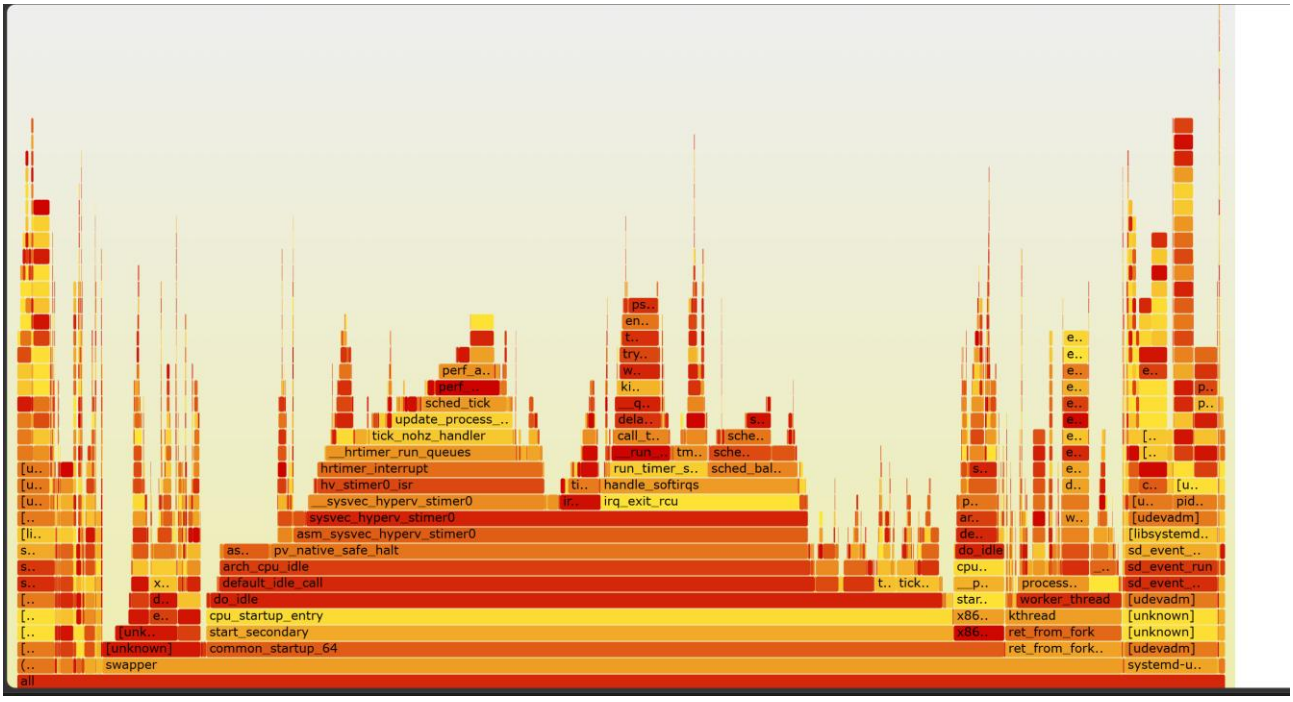
Лабораторна робота №2
Желдакова Анастасія Олександрівна
група тк-31

1. Побудувати FlameGraph виконання для системи

1.1. Процес побудови:

```
# git clone https://github.com/brendangregg/FlameGraph
# cd FlameGraph
# perf record -F 99 -a -g -- sleep 60
# perf script | ./stackcollapse-perf.pl > out.perf-folded
# ./flamegraph.pl out.perf-folded > perf.svg
# explorer.exe perf.svg
```

```
wo@DESKTOP-H3CUCMF:~/laba2$ git clone https://github.com/brendangregg/FlameGraph
Cloning into 'FlameGraph'...
remote: Enumerating objects: 1291, done.
remote: Counting objects: 100% (747/747), done.
remote: Compressing objects: 100% (163/163), done.
remote: Total 1291 (delta 614), reused 584 (delta 584), pack-reused 544 (from 1)
Receiving objects: 100% (1291/1291), 1.92 MiB | 1.04 MiB/s, done.
Resolving deltas: 100% (767/767), done.
wo@DESKTOP-H3CUCMF:~/laba2$ cd FlameGraph
wo@DESKTOP-H3CUCMF:~/laba2/FlameGraph$ sudo perf record -F 99 -a -g -- sleep 60
[ perf record: Woken up 1 times to write data ]
Failed to open /proc/schedstat
[ perf record: Captured and wrote 0.369 MB perf.data (1634 samples) ]
wo@DESKTOP-H3CUCMF:~/laba2/FlameGraph$ sudo perf script | ./stackcollapse-perf.pl > out.perf-folded
wo@DESKTOP-H3CUCMF:~/laba2/FlameGraph$ ./flamegraph.pl out.perf-folded > perf.svg
wo@DESKTOP-H3CUCMF:~/laba2/FlameGraph$ explorer.exe perf.svg
```



1.2 Аналіз системного FlameGraph

Вертикаль - показує глибину вкладеності. Чим вище функція на графіку, тим пізніше вона була викликана в ланцюжку.

Горизонталь - ширина кожного прямокутника показує кількість часу, який ця функція використовувала. Чим ширший прямокутник, тим більше часу пішло на виконання цієї функції.

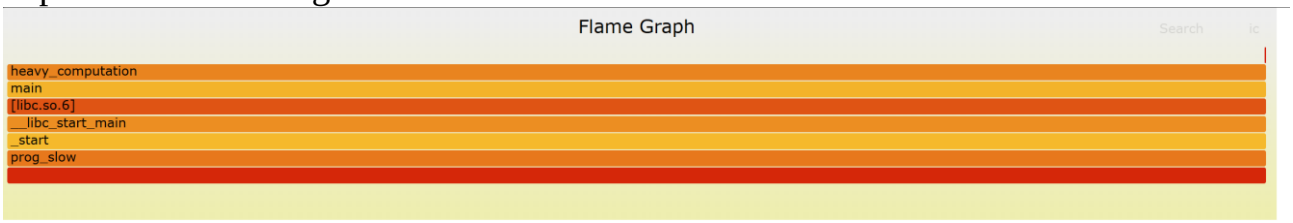
Результати:

- swapper / do_idle / common_startup_64: Найширший блок в основі графіка,який показує, що більшість часу протягом 60 секунд процесор перебував у стані спокою , адже важкі обчислювальні задачі в системі не виконувалися.
- hv_stimer0_isr / sysvec_hyperv_stimer0 - в центральній частині - це таймери віртуалізації WSL2, які підтримують зв'язок Linux із Windows.
- perf / perf_a..: щоб записати, що робить комп'ютер, сама perf має постійно працювати у фоні. Цей блок показує ресурси, які витратила програма perf на те, щоб спостерігати за системою.
- systemd-u.. / udevadm: це звичайні службові процеси Linux, які перевіряють пристрої і стежать за порядком у системі.

Висновок: Більшу частину часу система очікувала нових задач, а мінімальне навантаження створювалося системними таймерами та записом через perf.

FlameGraph для досліджуваної програми prog_slow

```
perf record -F 99 -g ./prog_slow
perf script | ./FlameGraph/stackcollapse-perf.pl > slow.folded
./FlameGraph/flamegraph.pl slow.folded > slow.svg
explorer.exe slow.svg
```



2. Зібрати та пояснити статистику виконання програми

2.1. /usr/bin/time --verbose ./prog_slow

```
wo@DESKTOP-H3CUCMF:~/laba2$ /usr/bin/time --verbose ./prog_slow
Done calculation! Result element: 1123875250.000000
        Command being timed: "./prog_slow"
        User time (seconds): 36.06
        System time (seconds): 0.13
        Percent of CPU this job got: 99%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:36.21
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 54664
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 13287
        Voluntary context switches: 1
        Involuntary context switches: 149
        Swaps: 0
        File system inputs: 0
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
```

- User time (seconds): 36.06: стільки секунд процесор витратив на виконання коду програми.
- System time (seconds): 0.13: стільки секунд процесор витратив на системні запити до ядра Linux (наприклад, виділення пам'яті).
- Percent of CPU this job got: 99%: відсоток завантаження ядра процесора. 99% означає, що програма завантажила одне ядро майже на максимуми.
- Elapsed (wall clock) time (h:mm:ss): 0:36.21: реальний час, який минув від моменту запуску програми до її завершення (36.21 секунди).
- Average shared/unshared text/data/stack/total size (kbytes): 0 - середні розміри різних ділянок пам'яті (коду, даних, стека)
- Maximum resident set size (kbytes): 54664: максимальний обсяг оперативної пам'яті, який використала програма.
- Average resident set size (kbytes): 0: середній обсяг використання RAM за весь час.
- Major (requiring I/O) page faults: 0: кількість звернень до пам'яті, які вимагали зчитування даних із жорсткого диска. Значення 0 означає, що програма не гальмувала через диск.
- Minor (reclaiming a frame) page faults: 13287: кількість виділень нових сторінок у RAM без використання диска.
- Voluntary context switches: 1: скільки разів програма добровільно віддавала процесор іншим задачам .
- Involuntary context switches: 149: скільки разів операційна система тимчасово забирала процесор у програми для інших системних потреб.
- Swaps: 0: програма не скидала дані на диск.
- File system inputs / outputs: 0 / 0: програма нічого не зчитувала і нічого не записувала на диск.
- Socket messages sent / received: 0 / 0: програма не працювала з мережею.
- Signals delivered: 0: програма не отримувала системних сигналів переривання.
- Page size (bytes): 4096: стандартний розмір однієї сторінки пам'яті в Linux.
- Exit status: 0: код завершення програми, успішно.

2.2 sudo perf stat -d ./prog_slow

```
wo@DESKTOP-H3CUCMF:~/laba2$ sudo perf stat -d ./prog_slow
[sudo: authenticate] Password:
Done calculation! Result element: 1123875250.000000

 Performance counter stats for './prog_slow':

            154      context-switches                 #      4.2 cs/sec  cs_per_second
              1      cpu-migrations                   #      0.0 migrations/sec  migrations_per_second
          13273      page-faults                      #    365.0 faults/sec  page_faults_per_second
       36360.43 msec task-clock                       #      1.0 CPUs  CPUs_utilized
     4309455447      L1-dcache-load-misses            #      7.5 %  l1d_miss_rate           (42.89%)
        7270568      branch-misses                    #      0.2 %  branch_miss_rate        (42.85%)
     3407709652      branches                         #     93.7 M/sec  branch_frequency     (57.14%)
    61124088675      cpu-cycles                       #      1.7 GHz  cycles_frequency      (57.11%)
   165921577989      instructions                     #      2.7 instructions  insn_per_cycle  (57.11%)
      565972342      stalled-cycles-frontend          #     0.01 frontend_cycles_idle       (57.15%)

    36.386258184 seconds time elapsed

    35.991756000 seconds user
     0.372369000 seconds sys


wo@DESKTOP-H3CUCMF:~/laba2$
```

- 154 context-switches - стільки разів операційна система перемикала процесор з програми на інші завдання і назад
- 1 cpu-migrations - програма 1 раз змінила процесорне ядро, на якому виконувалася.
- 13273 page-faults - звернення до оперативної пам’яті для отримання нових сторінок під дані .
- 36360.43 msec task-clock - сумарний час у мілісекундах, який програма фактично займала процесор. Позначка 1.0 CPUs utilized показує, що вона повністю утилізувала 1 ядро процесора.
- 4309455447 L1-dcache-load-misses (7.5% l1d_miss_rate) - у 7.5% випадків процесор не знаходив потрібні для роботи дані у своєму найшвидшому кеші, тому йому доводилося витрачати час і завантажувати їх із повільнішої пам'яті.
- 3407709652 branches (93.7 M/sec) - загальна кількість умовних переходів у коді (цикли, оператори if).
- 7270568 branch-misses (0.2% branch_miss_rate)- помилки передбачення розгалужень.
- 61124088675 cpu-cycles (1.7 GHz) - кількість тактів (коливань) процесора за час виконання програми.
- 165921577989 instructions (2.7 insn_per_cycle)- кількість виконаних процесорних інструкцій.
- 565972342 stalled-cycles-frontend (0.01 frontend_cycles_idle) - такти, коли процесор простоював у очікуванні подачі нових інструкцій.
-

Загальний час роботи:

- 36.386258184 seconds time elapsed - загальний час виконання (36.38 сек).
- 35.991756000 seconds user - час у просторі користувача (на наші обчислення).
- 0.372369000 seconds sys - час на системні виклики.

2.3 perf report (after perf record)

- perf record -g ./prog_slow
- perf report

```
Samples: 63K of event 'cpu/cycles/P', Event count (approx.): 61019447664
  Children      Self  Command    Shared Object          Symbol
+  100.00%     0.00%  prog_slow  prog_slow              [.] _start
+  100.00%     0.00%  prog_slow  libc.so.6             [.] __libc_start_main
+  100.00%     0.00%  prog_slow  libc.so.6             [.] 0x00007ef17c82a601
+  100.00%     0.00%  prog_slow  prog_slow              [.] main
+  100.00%    99.41%  prog_slow  prog_slow              [.] heavy_computation
      0.40%     0.04%  prog_slow  [kernel.kallsyms]     [.] asm_sysvec_hyperv_stimer0
      0.36%     0.00%  prog_slow  [kernel.kallsyms]     [.] sysvec_hyperv_stimer0
      0.32%     0.01%  prog_slow  [kernel.kallsyms]     [.] hv_stimer0_isr
      0.32%     0.00%  prog_slow  [kernel.kallsyms]     [.] __sysvec_hyperv_stimer0
      0.31%     0.00%  prog_slow  [kernel.kallsyms]     [.] hrtimer_interrupt
      0.27%     0.00%  prog_slow  [kernel.kallsyms]     [.] __hrtimer_run_queues
      0.25%     0.00%  prog_slow  [kernel.kallsyms]     [.] tick_nohz_handler
      0.19%     0.00%  prog_slow  [kernel.kallsyms]     [.] update_process_times
      0.15%     0.01%  prog_slow  [kernel.kallsyms]     [.] asm_exc_page_fault
      0.14%     0.01%  prog_slow  [kernel.kallsyms]     [.] exc_page_fault
      0.13%     0.00%  prog_slow  [kernel.kallsyms]     [.] do_user_addr_fault
      0.13%     0.00%  prog_slow  [kernel.kallsyms]     [.] sched_tick
      0.12%     0.00%  prog_slow  [kernel.kallsyms]     [.] handle_mm_fault
      0.12%     0.01%  prog_slow  [kernel.kallsyms]     [.] __handle_mm_fault
      0.10%     0.01%  prog_slow  [kernel.kallsyms]     [.] do_anonymous_page
      0.07%     0.01%  prog_slow  [kernel.kallsyms]     [.] task_tick_fair
      0.07%     0.00%  prog_slow  libc.so.6             [.] 0x00007ef17c8b3dbf
      0.05%     0.00%  prog_slow  libc.so.6             [.] 0x00007ef17c8b2ff4
      0.05%     0.01%  prog_slow  [kernel.kallsyms]     [.] tick_do_update_jiffies64
      0.04%     0.00%  prog_slow  [kernel.kallsyms]     [.] vma_alloc_folio_noprof
      0.04%     0.00%  prog_slow  [kernel.kallsyms]     [.] alloc_pages_mpol
      0.04%     0.00%  prog_slow  [kernel.kallsyms]     [.] __alloc_frozen_pages_noprof
      0.04%     0.00%  prog_slow  [kernel.kallsyms]     [.] update_wall_time
Cannot load tips.txt file, please install perf!
```

- Samples: 63K of event 'cpu/cycles/P' --- perf зробив 63 404 «знімки» (семпли) стану процесора за час роботи програми.
- Event count (approx.): 61019447664 - приблизна загальна кількість процесорних тактів, витрачених під час виконання програми.

Результати :

На функцію heavy_computation припадає 99.41% власного часу процесора (Self) та 100.00% загального часу з урахуванням дочірніх викликів (Children).

Системні виклики ядра Linux (kernel.kallsyms) та таймери віртуалізації WSL2 зайняли менше 1% часу.

3. Заміряти енерговитрати (Power consumption):

У WSL2 немає доступу до датчиків енергії, тому потужність виміряно у Windows програмою HWiNFO64 (показник «Повна споживана потужність ЦП», датчик процесора). Енергія E = P × t.

3.1 . Системи при виконанні програми

Простій системи: P = 5,662 Вт(середнє значення на рис.1)



Рисунок 1

Під час роботи: P = 10,265 Вт (на рис. 2)



Рисунок 2

Час роботи програми = 22,49 с

E_системи = 10,265 × 22,49 ≈ 230,9 Дж

3.2) виключно досліджуваної програми

E_програми = (10,265 − 5,662) × 22,49 = 4,603 × 22,49 ≈ 103,5 Дж

4. Порівняти параметри виконання програми до та після оптимізації

- - O3

```
.L4:
        mov     edi, 12000
        call    "malloc"
        mov     edi, 12000
        mov     QWORD PTR [rbp+0+r15*8], rax
        mov     r13, rax
        call    "malloc"
        mov     esi, 1
        mov     edi, 12000
        mov     QWORD PTR [rbx+r15*8], rax
        mov     r12, rax
        call    "calloc"
        movdqa  xmm5, XMMWORD PTR .LC0[rip]
        movdqa  xmm2, XMMWORD PTR .LC1[rip]
        movd    xmm6, r15d
        mov     QWORD PTR [r14+r15*8], rax
        lea     rdx, [r13+12000]
        mov     rax, r13
        pshufd  xmm3, xmm6, 0
        movdqa  xmm1, xmm5
.L2:
        movdqa  xmm0, xmm3
        add     rax, 32
        paddd   xmm0, xmm1
        paddd   xmm1, xmm2
        cvtdq2pd        xmm4, xmm0
        pshufd  xmm0, xmm0, 238
        movups  XMMWORD PTR [rax-32], xmm4
        cvtdq2pd        xmm0, xmm0
        movups  XMMWORD PTR [rax-16], xmm0
        cmp     rdx, rax
        jne     .L2
        mov     rax, r12
        lea     rdx, [r12+12000]
        movdqa  xmm1, xmm5
.L3:
        movdqa  xmm0, xmm3
        add     rax, 32
        psubd   xmm0, xmm1
        paddd   xmm1, xmm2
        cvtdq2pd        xmm4, xmm0
```

```
        pshufd  xmm0, xmm0, 238
        movups  XMMWORD PTR [rax-32], xmm4
        cvtdq2pd        xmm0, xmm0
        movups  XMMWORD PTR [rax-16], xmm0
        cmp     rdx, rax
        jne     .L3
        add     r15, 1
        cmp     r15, 1500
        jne     .L4
        xor     r8d, r8d
.L5:
        mov     rdi, QWORD PTR [r14+r8]
        mov     rsi, QWORD PTR [rbp+0+r8]
        xor     ecx, ecx
.L9:
        movsd   xmm1, QWORD PTR [rdi+rcx]
        xor     eax, eax
.L6:
        mov     rdx, QWORD PTR [rbx+rax]
        movsd   xmm0, QWORD PTR [rdx+rcx]
        mulsd   xmm0, QWORD PTR [rsi+rax]
        add     rax, 8
        addsd   xmm1, xmm0
        cmp     rax, 12000
        jne     .L6
        movsd   QWORD PTR [rdi+rcx], xmm1
        add     rcx, 8
        cmp     rcx, 12000
        jne     .L9
        add     r8, 8
        cmp     r8, 12000
        jne     .L5
        mov     rax, QWORD PTR [r14]
        mov     edi, OFFSET FLAT:.LC2
        movsd   xmm0, QWORD PTR [rax]
        add     rsp, 8
        mov     eax, 1
        pop     rbx
        pop     rbp
        pop     r12
        pop     r13
        pop     r14
        pop     r15
        jmp     "printf"
```

- –O0

```
.L9:
        mov     eax, DWORD PTR [rbp-28]
        cdqe
        lea     rdx, [0+rax*8]
        mov     rax, QWORD PTR [rbp-64]
        add     rax, rdx
```

```
        mov     rax, QWORD PTR [rax]
        mov     edx, DWORD PTR [rbp-32]
        movsxd  rdx, edx
        sal     rdx, 3
        add     rax, rdx
        movsd   xmm1, QWORD PTR [rax]
        mov     eax, DWORD PTR [rbp-28]
        cdqe
        lea     rdx, [0+rax*8]
        mov     rax, QWORD PTR [rbp-48]
        add     rax, rdx
        mov     rax, QWORD PTR [rax]
        mov     edx, DWORD PTR [rbp-36]
        movsxd  rdx, edx
        sal     rdx, 3
        add     rax, rdx
        movsd   xmm2, QWORD PTR [rax]
        mov     eax, DWORD PTR [rbp-36]
        cdqe
        lea     rdx, [0+rax*8]
        mov     rax, QWORD PTR [rbp-56]
        add     rax, rdx
        mov     rax, QWORD PTR [rax]
        mov     edx, DWORD PTR [rbp-32]
        movsxd  rdx, edx
        sal     rdx, 3
        add     rax, rdx
        movsd   xmm0, QWORD PTR [rax]
        mulsd   xmm0, xmm2
        mov     eax, DWORD PTR [rbp-28]
        cdqe
        lea     rdx, [0+rax*8]
        mov     rax, QWORD PTR [rbp-64]
        add     rax, rdx
        mov     rax, QWORD PTR [rax]
        mov     edx, DWORD PTR [rbp-32]
        movsxd  rdx, edx
        sal     rdx, 3
        add     rax, rdx
        addsd   xmm0, xmm1
        movsd   QWORD PTR [rax], xmm0
        add     DWORD PTR [rbp-36], 1
.L8:
        cmp     DWORD PTR [rbp-36], 1499
        jle     .L9
        add     DWORD PTR [rbp-32], 1
.L7:
        cmp     DWORD PTR [rbp-32], 1499
        jle     .L10
        add     DWORD PTR [rbp-28], 1
.L6:
        cmp     DWORD PTR [rbp-28], 1499
        jle     .L11
```

```
mov     rax, QWORD PTR [rbp-64]
mov     rax, QWORD PTR [rax]
mov     rax, QWORD PTR [rax]
movq    xmm0, rax
mov     edi, OFFSET FLAT:.LC1
mov     eax, 1
call    "printf"
nop
mov     rbx, QWORD PTR [rbp-8]
leave
ret
```

4. 1. Відмінності в асемблерному коді до/після оптимізації:

1. Робота з пам’яттю та стеком:
   - Без оптимізації (-O0, блок .L9): На кожному кроці циклу програма постійно читає змінні з оперативної пам’яті, а потім одразу записує результат назад. Через це виконується купа зайвих команд movsd.
   - З оптимізацією (-O3, блок .L6): Компілятор переніс усі лічильники та адреси масивів у швидкі регістри процесора (rdx, rcx, rax). Постійні звернення до пам’яті у внутрішньому циклі зникли.
2. Кількість інструкцій:

   У -O3 компілятор прибрав зайве, розгорнув цикли й зменшив кількість стрибків, завдяки чому загальна кількість виконаних інструкцій у perf stat впала з 165.9 млрд до 23.65 млрд (у 7 разів).

4.1.1. Приклад інструкцій AVX- розширень (SIMD)*

-Скалярна обробка (-O0 / -O3 у блоці .L6): Використовуються звичайні скалярні інструкції з суфіксом sd (наприклад, mulsd та addsd). Вони рахують лише по одному числу double за один крок процесора.
-Векторна обробка SIMD/AVX (-O3 у блоці .L2): Для обробки масивів компілятор задіяв векторні інструкції.

Приклад з коду (блок .L2):

```
add      rax, 32                          ; крок циклу одразу на 32 байти
paddd    xmm0, xmm1                       ; векторне додавання
cvtdq2pd xmm4, xmm0                       ; перетворення цілих чисел у double
movups   XMMWORD PTR [rax-32], xmm4       ; векторний запис
```

Інструкція cvtdq2pd перетворює цілий вектор чисел за один раз. А рядок add rax, 32 показує, що за одну ітерацію циклу обробляється одразу 32 байти (тобто 4 елементи double по 8 байт паралельно).

4.2 Порівняти час та інші показники

Завдяки тому, що компілятор переніс змінні в регістри процесора і прибрав зайві звернення до пам'яті (49 → 7 інструкцій на одну ітерацію циклу),

час виконання програми зменшився з 23,41 с (у prog_slow) до 11,14 с (у prog_fast). Програма почала працювати в 2,1 раза швидше.

```
wo@DESKTOP-H3CUCMF:~/laba2$ /usr/bin/time -v ./prog_fast
Done calculation! Result element: 1123875250.000000
        Command being timed: "./prog_fast"
        User time (seconds): 9.90
        System time (seconds): 0.10
        Percent of CPU this job got: 99%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:10.01
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 54776
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 13388
        Voluntary context switches: 1
        Involuntary context switches: 42
        Swaps: 0
        File system inputs: 0
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
```

```
wo@DESKTOP-H3CUCMF:~/laba2$ perf stat -d ./prog_fast
Done calculation! Result element: 1123875250.000000

 Performance counter stats for './prog_fast':

              0      context-switches:u                #      0.0 cs/sec  cs_per_second
              0      cpu-migrations:u                  #      0.0 migrations/sec  migrations_per_second
          13370      page-faults:u                     #   1201.3 faults/sec  page_faults_per_second
       11129.52 msec task-clock:u                      #      1.0 CPUs  CPUs_utilized
     4291414023      L1-dcache-load-misses:u           #     42.2 %  l1d_miss_rate          (42.87%)
        2422958      branch-misses:u                   #      0.1 %  branch_miss_rate       (42.90%)
     3374974881      branches:u                        #    303.2 M/sec  branch_frequency     (57.18%)
    32027917119      cpu-cycles:u                      #      2.9 GHz  cycles_frequency      (57.21%)
    23629086860      instructions:u                    #      0.7 instructions  insn_per_cycle  (57.13%)
       94970961      stalled-cycles-frontend:u         #     0.00 frontend_cycles_idle       (57.10%)

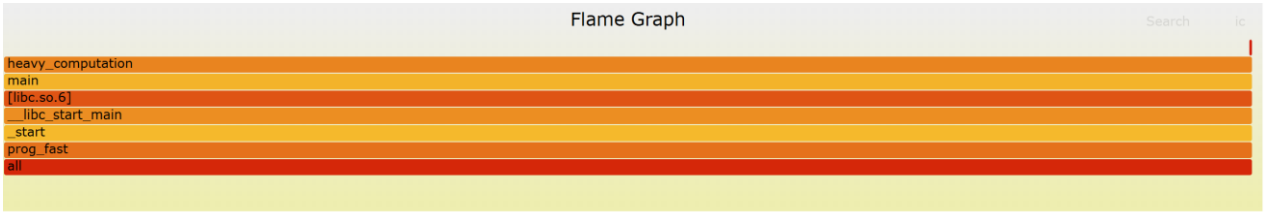
    11.139766424 seconds time elapsed

    11.048644000 seconds user
     0.084035000 seconds sys
```

Метрика	Повільна версія (prog_slow)	Оптимізована версія (prog_fast)	Зміна
Реальний час (Elapsed time)	23,41	11,14	менше в 2,1 раза
User CPU Time	23,32	11,05	менше в 2,1 раза
Кількість інструкцій (instructions)	165,42 млрд	23,63 млрд	менше в 7 разів

Метрика	Повільна версія (prog_slow)	Оптимізована версія (prog_fast)	Зміна
Такти CPU (cpu-cycles)	72,73 млрд	32,03 млрд	менше в 2,3 раза
Швидкість обчислень (IPC)	2,3 insn/cycle	0,7 insn/cycle	знизився в 3 рази: процесор чекає дані з пам'яті

4.3 Зміни на FlameGraph (п. 1) після оптимізації



Що змінилось:

1. Кількість семплів (samples). Для prog_slow - 68 млрд, для prog_fast - 24 млрд, тобто приблизно в 2,8 раза менше. При частоті 99 знімків на секунду це відповідає скороченню часу роботи (36 с → 14 с). Сама форма графіка однакова, але в prog_fast значно менше часу роботи процесора.
2. Вузький блок праворуч (робота ядра: запуск програми, виділення пам'яті через malloc/calloc, page faults) став відносно ширшим. Ця робота не змінилась, бо матриці того самого розміру, а час обчислень зменшився, тож її частка зросла.

FlameGraph показує, де програма витрачає час, а не скільки. Тому після оптимізації його форма майже не змінилась. Прискорення видно за кількістю семплів, яка зменшилась пропорційно часу виконання.

4.4) Порівняння енерговитрат та пояснення причин:

Розрахунок для prog_fast

$P = 10{,}351$ Вт

$t = 10{,}01$ с

$E_{прог} = (10{,}351 - 5{,}662) \times 10{,}01 = 4{,}689 \times 10{,}01 \approx 46{,}9$ Дж

	prog_slow (-O0)	prog_fast (-O3)	**Зміна**
Середня потужність, Вт	10,265	10,351	майже не змінилась
Час виконання, с	22,49	10,01	менше в **2,25 раза**
Енергія системи, Дж	230,9	103,6	менше в **2,23 раза**
Енергія програми, Дж	103,5	46,9	менше в **2,21 раза**

Головна перевага в енергоспоживанні досягається не за рахунок зміни потужності (Вт), а за рахунок скорочення часу роботи під навантаженням (з 22,49 с до 10,01 с).

А завдяки оптимізації компілятора -O3 (перенесення змінних у регістри, 49 → 7 інструкцій на ітерацію) програма виконує той самий обсяг обчислень за значно меншу кількість тактів процесора і дозволяє швидше вивільнити ресурси та повернутися у фоновий стан спокою.